LoopForge — Implementation Document

Nov 6, 2026 · @Madhavi

LoopForge is an autonomous, feedback-driven AI software engineering platform that turns natural-language requirements into tested Python applications by iterating until measurable acceptance criteria are met. This document specifies how to build it, phase by phase.

Field	Detail
Project	LoopForge — An Autonomous, Feedback-Driven AI Software Engineering Platform
Document type	Implementation specification
Version	1.0
Prepared by	Madhavi
Delivery model	Phase-gated (six phases, each closed by an exit gate)
Discipline	Agentic AI and loop engineering

1. Overview

1.1 Problem statement

AI coding assistants can generate code, explain errors and suggest fixes, but generating code does not prove the application works. Six recurring gaps motivate LoopForge:

1. Generated code may contain syntax and runtime errors.
2. Agents may fix one defect while introducing another.
3. Implementations may not satisfy every requirement.
4. Agents may claim success without execution evidence.
5. Repeated correction attempts can consume excessive tokens and compute.
6. A single agent judging its own output often misses its own mistakes.

1.2 Proposed solution

LoopForge accepts a development task, decomposes it into objectives, generates implementation files, executes automated tests in a sandbox, evaluates the results independently and decides whether another iteration is needed. It is not a wrapper around an LLM API; it is an engineered development lifecycle with:

- Multi-agent orchestration
- Iterative feedback and self-correction
- Independent code verification
- Persistent execution state
- Controlled, sandboxed code execution
- Quantitative performance evaluation
- Human oversight and auditability

Every iteration is recorded: generated code, test results, error logs, evaluation reports, changes introduced, token consumption, execution time and final acceptance status.

1.3 MVP scope

Component	MVP scope
Programming language	Python
Application type	REST APIs and backend applications
Framework	FastAPI
Testing	Pytest
Code quality	Ruff
Security analysis	Bandit
Execution	Isolated Docker containers
Agent framework	LangGraph
LLM	OpenAI API
Interface	React dashboard

Out of scope for the MVP: JavaScript and Java targets, additional application types, and any unrestricted access to the host filesystem or terminal. LoopForge generates and modifies projects only inside its own isolated workspace.

2. Design Principles and Key Decisions

The evaluation engine, not the number of agents, is the centrepiece of LoopForge. Four well-designed agents with strong verification are worth more than twelve agents passing messages to one another.

2.1 Guiding principles

1. **Evidence over assertion.** Success is established only by executed tests and deterministic checks, never by an agent's claim.
2. **Separation of maker and checker.** The agent that writes or repairs code never approves its own work.
3. **Deterministic control.** Loop decisions (retry, accept, escalate, stop) are made by plain Python code, not by an LLM.
4. **Revalidate everything.** Every modified implementation returns to execution and testing, never directly to approval.
5. **Bounded by default.** Iterations, time, tokens and cost are capped from the first run.
6. **Isolation first.** Generated code runs only inside a restricted sandbox.
7. **Full traceability.** Every iteration is versioned, with diffs, test evidence and agent explanations.
8. **Loop before interface.** The working feedback loop is built before the dashboard.

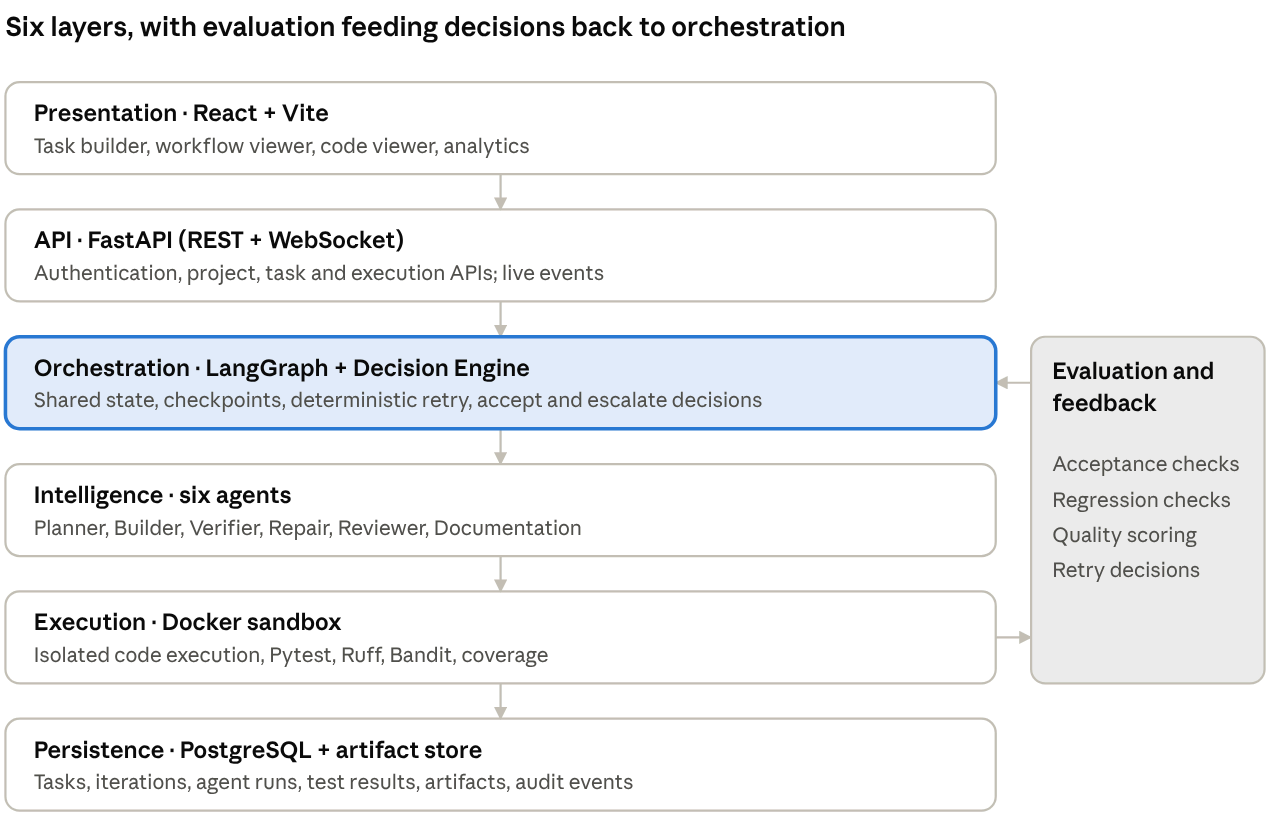
2.2 Architecture decision record

Decision	Choice	Rationale
Architecture style	Modular monolith	Simpler to build, debug and test; a separate worker service can be split out later
Agent framework	LangGraph	Explicit graph control flow, shared state and checkpointing

Decision	Choice	Rationale
First supported language	Python	Mature tooling for testing, linting and security scanning
Execution environment	Docker sandbox	Isolation, resource limits and reproducibility
Evaluation strategy	Independent tests plus independent reviewer	Prevents self-grading and false success
Loop controller	Deterministic Python component	Predictable, auditable termination
Database	PostgreSQL with SQLAlchemy 2.x and Alembic	Relational integrity plus JSONB for flexible agent output
Deferred components	Redis, Kafka, Kubernetes, vector databases	Add complexity without solving an MVP problem

3. System Architecture

LoopForge is a modular monolith of six layers. Execution results flow into the evaluation and feedback layer, which returns control to the appropriate agent through the orchestrator whenever validation fails.



system architecture · 6 layers + evaluation loop

Layer	Responsibility	Key components
Presentation	User interaction and monitoring	React + Vite, React Flow, Recharts
API	Authentication, task management, communication	FastAPI, REST, WebSocket
Orchestration	Agent coordination and workflow execution	LangGraph engine, Decision Engine, checkpointer
Intelligence	Planning, coding, testing, debugging, review, documentation	Six agents (Section 4)
Execution	Sandboxed code execution and testing	Docker, Pytest, Ruff, Bandit, Coverage.py
Persistence	Tasks, iterations, artifacts, evaluation history	PostgreSQL, artifact storage

The evaluation and feedback layer spans orchestration and execution: it performs acceptance checks, regression checks, quality scoring and supplies the evidence the Decision Engine uses for retry decisions.

4. Agent Specifications

LoopForge uses six logical agents and one deterministic controller. Operations that need predictable behaviour are ordinary Python functions, not LLM calls.

Agent	Role	Input	Output	May not
Planner	Requirement analysis and planning	Natural-language task	Validated project plan with acceptance criteria	Write code
Builder	Code generation	Approved plan	Structured file changes	Run shell commands freely
Verifier	Testing	Generated workspace	Machine-readable test report	Infer success without execution
Repair	Debugging and correction	Failure report, relevant files	Patch plus explanation	Mark its own fix as successful
Reviewer	Code review and quality	Workspace, plan, tool reports	Review report with critical issues and requirement coverage	Rely only on LLM scores
Documentation	Final documentation	Accepted or terminated task	README, API docs, iteration summary	Run before a terminal state
Decision Engine	Loop control (deterministic)	Test and review results, budgets	Next action: retry, complete, escalate, stop	Use an LLM to decide

4.1 Planner Agent (requirement analysis and planning)

Responsibilities: parse requirements; separate functional and non-functional requirements; determine files, modules and dependencies; define acceptance criteria; produce an execution plan.

Example input: "Create an employee management API with authentication, CRUD operations and unit tests."

Example output (validated against a strict Pydantic schema):

```
{
  "project_name": "employee_management",
  "framework": "FastAPI",
  "requirements": [
    "User authentication",
    "Create employee",
    "Read employee",
    "Update employee",
    "Delete employee",
    "Unit tests"
  ],
  "acceptance_criteria": [
    "All API endpoints respond correctly",
    "Authentication is enforced",
    "Invalid input is rejected",
    "All required tests pass"
  ]
}
```

4.2 Builder Agent (code generation)

Responsibilities: create the project structure; generate Python source files and `requirements.txt`; implement REST endpoints and data validation; generate initial tests. It emits structured file changes rather than shell commands.

```
app/
    main.py
    models.py
    schemas.py
    routes.py
    auth.py
tests/
    test_routes.py
requirements.txt
```

4.3 Verifier Agent (testing)

Responsibilities: run Pytest and API tests; check syntax; collect failed test names and stack traces; record execution time; emit a machine-readable report. It reports actual execution results only.

```
{
  "total_tests": 10,
  "passed": 8,
  "failed": 2,
  "status": "FAILED",
  "failures": [
    {
      "test": "test_employee_update",
      "reason": "Expected HTTP 200, received 404"
    }
  ]
}
```

4.4 Repair Agent (debugging and correction)

Responsibilities: analyse failures; identify the likely root cause; inspect relevant files; apply a correction; explain what changed and why. Its output always routes back to the Verifier.

```
Detected issue:  Employee update endpoint returns HTTP 404.
Root cause:      The route queries the wrong employee identifier.
Correction:      Database lookup now uses employee_id.
Next action:     Re-run affected tests and the full regression suite.
```

4.5 Reviewer Agent (code review and quality)

Responsibilities: review structure; detect duplicate logic; flag insecure practices and missing validation; assess maintainability and coverage; check for omitted requirements. It combines LLM review with deterministic tools: Ruff (linting), Bandit (static security) and Coverage.py (coverage).

4.6 Documentation Agent

Responsibilities: generate README.md, API endpoint documentation, architecture and setup notes, test results and an iteration summary. Runs only after the task is accepted or explicitly terminated.

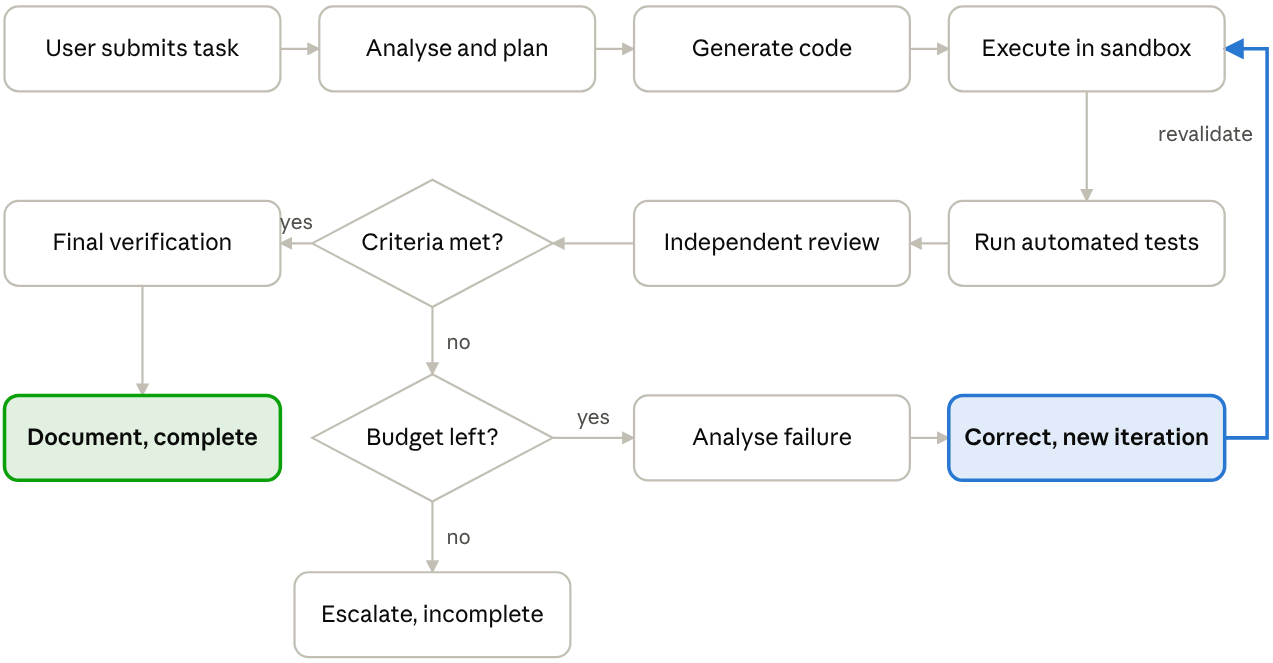
4.7 Decision Engine (deterministic controller)

The Decision Engine is not an agent. It reads test and review results; checks acceptance criteria; decides whether a retry is justified; enforces iteration, token and time budgets; detects repeated failures; requests human intervention; and stops the workflow when the task is accepted or cannot safely continue. Keeping this logic deterministic is essential to reliability.

5. Loop Engine

The loop engine is the core of LoopForge: a genuine feedback cycle, not a sequence of agents that each run once. After any correction the workflow returns to execution and testing, so every modified implementation is revalidated before it can be accepted.

Every correction returns to execution, never straight to approval



execution lifecycle · 2 decisions, 2 exits

5.1 Shared state

LangGraph passes one typed state object between nodes. Large files and logs are stored as artifacts and referenced by ID rather than passed repeatedly through the LLM context.

```
from typing import TypedDict, Optional, List, Dict, Any

class LoopForgeState(TypedDict):
    task_id: str
    user_requirement: str
    project_plan: Dict[str, Any]
    generated_files: Dict[str, str]        # path -> artifact id
    test_results: Dict[str, Any]
    review_results: Dict[str, Any]
    failure_analysis: Optional[str]
    failure_fingerprints: List[str]        # for repeated-failure detection
    iteration: int
    max_iterations: int
    tokens_used: int
    started_at: float
    execution_history: List[Dict[str, Any]]
    quality_score: float
    status: str
    final_report: Optional[Dict[str, Any]]
```

A LangGraph checkpointer persists state after every node so a workflow can resume after interruption.

5.2 Termination configuration

Starting values, tuned during Phase 6 benchmarking; they are not universal standards.

Parameter	Initial value
Maximum iterations	5
Maximum execution time per iteration	120 seconds
Maximum total task duration	15 minutes
Maximum correction attempts per distinct failure	2
Token budget per task	30,000
Minimum test pass rate	100%
Security	No unapproved high-severity findings

5.3 Decision Engine logic

Acceptance requires structured evidence against every acceptance criterion; a zero-failure test suite alone does not prove the requirements are met.

```
import hashlib, time
from enum import Enum

class Decision(str, Enum):
    COMPLETE = "complete"
    RETRY = "retry"
    ESCALATE = "escalate"
    STOP = "stop"

def failure_fingerprint(tests: dict) -> str:
    keys = sorted(f"{f['test']}:{f.get('error_type', '')}" for f in
tests["failures"])
    return hashlib.sha256("|".join(keys).encode()).hexdigest()

def evaluate_iteration(state: dict, limits) -> Decision:
    if state["status"] == "cancelled":
        return Decision.STOP

    tests, review = state["test_results"], state["review_results"]
    accepted = (
        tests["total_tests"] > 0
        and tests["failed"] == 0
        and review["critical_issues"] == 0
        and review["unapproved_high_severity"] == 0
        and all(c["met"] for c in review["acceptance_criteria"])
    )
    if accepted:
        return Decision.COMPLETE

    if state["iteration"] >= state["max_iterations"]:
        return Decision.ESCALATE
    if state["tokens_used"] >= limits.max_tokens:
        return Decision.ESCALATE
    if time.time() - state["started_at"] >= limits.max_task_seconds:
        return Decision.ESCALATE

    fp = failure_fingerprint(tests)
    if state["failure_fingerprints"].count(fp) >=
```

```
limits.max_attempts_per_failure:
        return Decision.ESCALATE        # same failure keeps recurring

    return Decision.RETRY
```

5.4 Feedback channels

Three separate channels make diagnosis precise; the error classifier tags each failure with its channel before it reaches the Repair Agent.

Channel	Question answered	Examples	Source
Execution feedback	What happened when the code ran?	Syntax, import and runtime errors; API failures	Sandbox, Pytest
Quality feedback	How good is the implementation?	Duplication, missing validation, vulnerabilities, low coverage	Ruff, Bandit, Coverage.py, Reviewer
Requirement feedback	Does it fulfil the task?	Missing endpoint or authentication, wrong response format, missing docs	Reviewer against acceptance criteria

5.5 Versioning and change tracking

Every iteration is a distinct version with file hashes, diffs, test reports and agent explanations, so users can see exactly how the implementation improved. Illustrative record:

Version	Change	Result
v1	Initial code generation	4 tests failed
v2	Corrected routing logic	2 tests failed
v3	Corrected validation	All tests passed
v4	Final review	Accepted

6. Module Breakdown

The system is divided into ten modules. Each maps to a package in the repository (Section 10) and is delivered in a specific phase (Section 11).

Module	Name	Responsibilities	Package	Phase
M1	User and Project Management	Registration, authentication, project and task creation, status tracking, project history	`api/`, `services/`	1
M2	Requirement Processor	Requirement extraction, functional requirements, acceptance criteria, dependencies, plan validation	`agents/planner.py`, `schemas/`	2
M3	Agent Orchestrator	LangGraph workflow, agent invocation, shared state, checkpointing, recovery	`orchestrator/`	2
M4	Code Generator	File creation and modification, dependency generation, source management	`agents/coder.py`	2

Module	Name	Responsibilities	Package	Phase
M5	Sandbox Executor	Container creation, dependency install, command execution, timeouts, log collection, cleanup	`execution/`	3
M6	Test and Validation Engine	Pytest, Ruff, Bandit, coverage, API validation	`evaluation/`	3
M7	Feedback and Correction Engine	Failure classification, error analysis, root cause, patch generation, regression prevention	`feedback/`, `agents/debugger.py`	4
M8	Loop Controller	Retry and acceptance decisions, budget enforcement, escalation, human approval	`orchestrator/controller.py`	4
M9	Analytics and Monitoring	Iteration, quality score, execution time, token and agent-level tracking	`services/analytics_service.py`, frontend	5

Module	Name	Responsibilities	Package	Phase
M10	Reporting and Documentation	README, API docs, final report, iteration comparison, downloadable artifacts	`agents/documentation.py`, `services/artifact_service.py`	6

7. Data Model

PostgreSQL is the primary store, with eight tables. Use SQLAlchemy 2.x and Alembic migrations; keep queryable fields as relational columns and use JSONB only for flexible agent output.

7.1 Entity relationships

Parent	Relationship	Child
users	owns (1:N)	projects
projects	contains (1:N)	tasks
tasks	executes (1:N)	iterations
tasks	records (1:N)	audit_events
iterations	contains (1:N)	agent_runs
iterations	produces (1:N)	test_results
iterations	generates (1:N)	artifacts

7.2 Table specifications

users

Column	Type	Notes
id	UUID	Primary key
email	VARCHAR	Unique

Column	Type	Notes
password_hash	VARCHAR	Hashed password
created_at	TIMESTAMP	Account creation

projects

Column	Type	Notes
id	UUID	Primary key
user_id	UUID	FK → users.id
name	VARCHAR	Project name
description	TEXT	Project description
language	VARCHAR	Python initially
created_at	TIMESTAMP	Creation date

tasks

Column	Type	Notes
id	UUID	Primary key
project_id	UUID	FK → projects.id
requirement	TEXT	User's task description
status	VARCHAR	See 7.3
max_iterations	INTEGER	Iteration limit
current_iteration	INTEGER	Current iteration
acceptance_criteria	JSONB	Required conditions
created_at	TIMESTAMP	Creation date

iterations

Column	Type	Notes
id	UUID	Primary key
task_id	UUID	FK → tasks.id
iteration_number	INTEGER	Sequence within task
status	VARCHAR	Iteration result
quality_score	FLOAT	Composite score (Section 12.4)
feedback	JSONB	Evaluation feedback
started_at	TIMESTAMP	Start time
completed_at	TIMESTAMP	Completion time

agent_runs — one row per agent execution

Column	Type	Notes
id	UUID	Primary key
iteration_id	UUID	FK → iterations.id
agent_name	VARCHAR	Planner, Builder, Verifier, Repair, Reviewer, Documentation
input_summary	TEXT	Summary, not full prompt
output_summary	TEXT	Summary of result
status	VARCHAR	Success, failed, timeout
tokens_used	INTEGER	Input plus output tokens
duration_seconds	FLOAT	Wall-clock duration
error_message	TEXT	If failed

test_results — execution evidence

Column	Type	Notes
id	UUID	Primary key

Column	Type	Notes
iteration_id	UUID	FK → iterations.id
total_tests	INTEGER	
passed	INTEGER	
failed	INTEGER	
skipped	INTEGER	
coverage	FLOAT	Percent
details	JSONB	Failures, stack traces
created_at	TIMESTAMP	

artifacts — references to generated files and reports

Column	Type	Notes
id	UUID	Primary key
iteration_id	UUID	FK → iterations.id
artifact_type	VARCHAR	Source, diff, log, report
file_name	VARCHAR	
storage_path	TEXT	Location on disk or object store
file_hash	VARCHAR	Content hash for versioning
created_at	TIMESTAMP	

audit_events — significant system events

Column	Type	Notes
id	UUID	Primary key
task_id	UUID	FK → tasks.id
event_type	VARCHAR	e.g. task_created, agent_invoked, sandbox_started, test_failed, retry_approved, task_completed

Column	Type	Notes
event_data	JSONB	Event payload
created_at	TIMESTAMP	

7.3 Task status lifecycle

`PENDING` → `PLANNING` → `GENERATING` → `EXECUTING` → `TESTING` → `REVIEWING` → (`REPAIRING` → back to `EXECUTING`) → terminal state.

Terminal states: `COMPLETED`, `FAILED`, `ESCALATED`, `CANCELLED`.

8. API Specification

The FastAPI backend exposes six REST groups under `/api/v1` and one WebSocket channel. Agent execution is controlled by the orchestrator; ordinary users never invoke agents directly.

Group	Method	Endpoint	Purpose	Phase
Auth	POST	`/api/v1/auth/register`	Register user	1
Auth	POST	`/api/v1/auth/login`	Log in	1
Auth	GET	`/api/v1/auth/me`	Current user	1
Projects	POST	`/api/v1/projects`	Create project	1
Projects	GET	`/api/v1/projects`	List projects	1
Projects	GET	`/api/v1/projects/{id}`	Project details	1
Projects	PUT	`/api/v1/projects/{id}`	Update project	1
Projects	DELETE	`/api/v1/projects/{id}`	Delete project	1
Tasks	POST	`/api/v1/tasks`	Submit task	1
Tasks	GET	`/api/v1/tasks`	List tasks	1
Tasks	GET	`/api/v1/tasks/{id}`	Task details	1
Tasks	POST	`/api/v1/tasks/{id}/start`	Start execution	2

Group	Method	Endpoint	Purpose	Phase
Tasks	POST	`/api/v1/tasks/{id}/cancel`	Cancel execution	4
Tasks	POST	`/api/v1/tasks/{id}/retry`	Retry a failed task	4
Iterations	GET	`/api/v1/tasks/{id}/iterations`	List iterations	4
Iterations	GET	`/api/v1/iterations/{id}`	Iteration details	4
Iterations	GET	`/api/v1/iterations/{id}/diff`	Code changes	4
Iterations	GET	`/api/v1/iterations/{id}/logs`	Execution logs	3
Agents	GET	`/api/v1/agents`	List agents	2
Agents	GET	`/api/v1/agents/{name}/status`	Agent status	5
Agents	GET	`/api/v1/tasks/{id}/agent-runs`	Agent execution history	4
Analytics	GET	`/api/v1/analytics/overview`	Overall metrics	5
Analytics	GET	`/api/v1/analytics/tasks/{id}`	Task metrics	5
Analytics	GET	`/api/v1/analytics/iterations/{id}`	Iteration performance	5
Analytics	GET	`/api/v1/analytics/agents`	Agent performance	5

8.1 Authentication

Use a secure password-hashing algorithm and JWT-based or secure session authentication. All endpoints other than register and login require authentication, and users may access only their own projects and tasks.

8.2 Real-time events (WebSocket)

Endpoint: `/ws/tasks/{task_id}`. The dashboard subscribes per task and updates without refreshing.

```
{
  "event": "iteration_completed",
  "task_id": "task-123",
  "iteration": 2,
  "status": "failed",
  "tests_passed": 8,
  "tests_failed": 2
}
```

Recommended event types: `task_started`, `agent_started`, `agent_completed`, `sandbox_started`, `tests_completed`, `iteration_completed`, `retry_scheduled`, `escalated`, `task_completed`, `task_cancelled`.

9. Frontend Specification

The React + Vite frontend is an engineering control centre, not a chatbot. It has seven screens.

#	Screen	Content	Primary APIs
1	Command Center	Active and completed tasks, average iterations, success rate, recent executions	`/analytics/overview`, `/tasks`
2	Task Creation	Project name, natural-language requirement, framework, acceptance criteria, iteration limit, execution budget	`POST /tasks`
3	Live Workflow	Agent execution graph (React Flow), current agent and iteration, workflow status, execution timeline	WebSocket, `/tasks/{id}/agent-runs`
4	Code Workspace	Generated file tree, code viewer, terminal logs, file changes, version comparison	`/iterations/{id}/diff`, `/iterations/{id}/logs`
5	Evaluation Center	Test results, code quality, coverage, security findings, acceptance criteria status	`/iterations/{id}`

#	Screen	Content	Primary APIs
6	Loop Analytics	Quality improvement, iterations per task, cost per task, failure distribution, agent performance	`/analytics/*`
7	Final Report	Final status, generated files, test and review summaries, iteration history, downloadable artifacts	`/tasks/{id}`, artifacts

9.1 Dashboard KPIs

KPI	Calculation
Task success rate	Completed tasks ÷ eligible finished tasks × 100
Average iterations	Total iterations ÷ completed tasks
Test pass rate	Passed tests ÷ executed tests × 100
Average quality score	Mean score of accepted tasks
Average execution time	Total execution duration ÷ completed tasks
Average cost	Total model and execution cost ÷ tasks

Metric definitions: cancelled tasks are excluded from the failure count, and test counts must not be double-counted when aggregating repeated runs.

10. Repository Structure

Repository name: `loopforge`. Agent definitions, orchestration, execution and evaluation live in separate packages so loop strategies can be changed without rewriting the application.

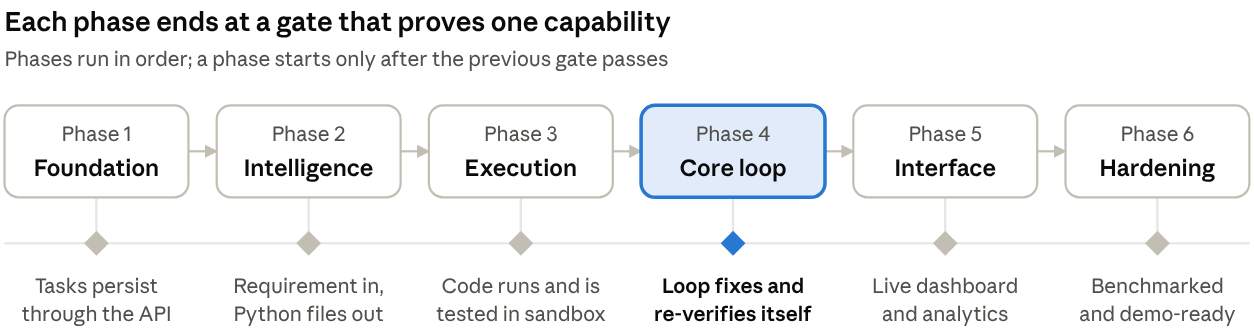
```
loopforge/
├── backend/
|   ├── app/
|   |   ├── main.py
|   |   ├── config.py
```

```
|   |   ├── api/
|   |   |   ├── dependencies.py
|   |   |   └── v1/  auth.py  projects.py  tasks.py  iterations.py  agents.py
analytics.py
|   |   ├── agents/        planner.py  coder.py  tester.py  debugger.py
reviewer.py  documentation.py
|   |   ├── orchestrator/  graph.py  state.py  nodes.py  edges.py
controller.py  checkpoints.py
|   |   ├── execution/     docker_manager.py  sandbox.py  command_runner.py
resource_limits.py
|   |   ├── evaluation/    test_runner.py  quality_checker.py
security_checker.py
|   |   |                  requirement_checker.py  scoring.py
|   |   ├── feedback/      error_classifier.py  root_cause.py
patch_manager.py
|   |   ├── database/      models.py  session.py  repositories.py
|   |   ├── schemas/       task.py  project.py  agent.py  evaluation.py
|   |   ├── services/      task_service.py  project_service.py
artifact_service.py  analytics_service.py
|   |   └── utils/         logging.py  security.py  token_tracker.py
|   ├── migrations/
|   ├── tests/         unit/  integration/  agents/  evaluation/
|   ├── requirements.txt
|   └── Dockerfile
├── frontend/
|   ├── src/
|   |   ├── components/    dashboard/  workflow/  code-editor/  analytics/
common/
|   |   ├── pages/         Dashboard.jsx  CreateTask.jsx  Workflow.jsx
Workspace.jsx
|   |   |                  Evaluation.jsx  Analytics.jsx  Report.jsx
|   |   ├── hooks/  services/  context/  layouts/
|   |   └── App.jsx
|   └── package.json
├── sandbox/           Dockerfile  execution_config/
├── benchmarks/        tasks/  expected_outputs/  evaluation_scripts/
results/
├── docs/              architecture.md  api_documentation.md
database_schema.md
|                      evaluation_methodology.md  security_model.md
├── scripts/           setup.sh  seed_database.py  run_benchmarks.py
├── docker-compose.yml
```

```
├── .env.example
├── .gitignore
├── README.md
└── LICENSE
```

11. Phase-wise Implementation Plan

Delivery runs in six sequential phases. Each phase has a defined objective, entry criteria, work packages, deliverables and an exit gate. Progress is measured by gates passed, not by elapsed time: a phase is complete only when its gate is demonstrated with evidence.



implementation roadmap · 6 phases, 6 exit gates

Sequencing rules

1. Build the execution loop before the interface. Do not start with the dashboard or with all six agents.
2. Phase 4's gate is the project's critical milestone: a working autonomous Python coding loop. A functioning loop outweighs any number of agents that do not work together.
3. Defects found against an earlier gate are fixed before the current phase continues.
4. Every phase adds tests for the code it introduces (Section 12.1).

11.1 Phase 1 — Foundation

Objective: a working backend that stores users, projects and tasks persistently.

Entry criteria: repository created; Docker Desktop, Python and PostgreSQL available locally.

Work package	Modules	Description
1.1 Project scaffolding	—	Repository layout (Section 10), `docker-compose.yml`, `.env.example`, linting and test setup
1.2 Database layer	M1	SQLAlchemy models for all eight tables; initial Alembic migration
1.3 Authentication	M1	Register, login, current-user endpoints; password hashing; JWT or secure sessions
1.4 Project and task APIs	M1	CRUD for projects; create, list and read tasks
1.5 Task status management	M1	Status enum and allowed transitions (Section 7.3); audit events on change

Deliverables: running FastAPI service; migrated database; authenticated project and task APIs; unit and integration tests.

Exit gate:

- [ ] A registered user can create a project and a task, and both persist across a service restart.
- [ ] Invalid status transitions are rejected.
- [ ] Users cannot read other users' projects.

11.2 Phase 2 — Agent Intelligence

Objective: turn a natural-language task into a validated plan and generated Python files.

Entry criteria: Phase 1 gate passed; OpenAI API key configured.

Work package	Modules	Description
2.1 LLM integration	M3	OpenAI SDK client with token tracking and retry on transient errors
2.2 Structured output schemas	M2	Pydantic schemas for plans and file changes
2.3 Planner Agent	M2	Requirements, acceptance criteria, file plan

Work package	Modules	Description
2.4 Builder Agent	M4	Structured file changes written to an isolated task workspace
2.5 LangGraph state and graph	M3	`LoopForgeState`, planner → builder nodes, task start endpoint

Deliverables: planner and builder agents; LangGraph graph with shared state; token usage recorded in `agent_runs`.

Exit gate:

- [ ] Submitting the demo requirement produces a schema-valid plan with acceptance criteria.
- [ ] The builder writes a complete project structure into the task workspace only.
- [ ] Malformed LLM output is rejected and logged, not silently accepted.

11.3 Phase 3 — Sandboxed Execution and Testing

Objective: execute generated code safely and collect real test evidence.

Entry criteria: Phase 2 gate passed.

Work package	Modules	Description
3.1 Sandbox image	M5	Python image with non-root user and restricted tooling
3.2 Docker manager	M5	Container lifecycle, mounts, CPU, memory and process limits, timeouts, no network, cleanup
3.3 Command runner	M5	Allowlisted commands; stdout, stderr and exit codes captured
3.4 Test runner	M6	Pytest execution with machine-readable report parsing
3.5 Verifier Agent	M6	Builds the test report from real execution only
3.6 Log storage	M10	Logs stored as artifacts; `/iterations/{id}/logs` endpoint

Deliverables: hardened sandbox; test reports stored in `test_results`; execution logs as artifacts.

Exit gate:

- [ ] Generated code runs only inside the sandbox, and every Section 13.1 control is verified.
- [ ] A deliberately broken file produces a captured failure with stack trace.
- [ ] A runaway process is killed at the time limit and the container is removed.

11.4 Phase 4 — Core Feedback Loop (critical milestone)

Objective: close the loop so that LoopForge diagnoses failures, corrects code and re-verifies automatically until acceptance or a limit.

Entry criteria: Phase 3 gate passed.

Work package	Modules	Description
4.1 Error classifier	M7	Classifies execution, quality and requirement failures (Section 5.4)
4.2 Repair Agent	M7	Root-cause analysis and patch generation via structured changes
4.3 Reviewer Agent and quality checks	M6	Ruff, Bandit, Coverage.py plus LLM requirement review
4.4 Decision Engine	M8	Deterministic retry, accept, escalate and stop logic; budgets; failure fingerprinting
4.5 Iteration history	M3, M10	Versioned iterations with diffs, hashes, reports; checkpoints; cancel, retry and diff endpoints

Deliverables: complete autonomous loop; iteration history in PostgreSQL; resumable workflows.

Exit gate (Milestone 1 — autonomous Python coding loop):

- [ ] A task with an injected defect fails, is repaired and passes on a later iteration without human input.
- [ ] Every repair is followed by a full test re-run before acceptance.

- [ ] The loop stops and escalates at the iteration, time or token limit.
- [ ] Identical repeated failures trigger escalation.
- [ ] A workflow interrupted mid-iteration resumes from its checkpoint.

11.5 Phase 5 — Interface and Analytics

Objective: make the loop visible and inspectable.

Entry criteria: Phase 4 gate passed.

Work package	Modules	Description
5.1 Frontend scaffold	M9	React + Vite, routing, authentication flow, API client
5.2 Task creation and command center	M9	Screens 1 and 2
5.3 Live workflow	M9	Screen 3 with React Flow and WebSocket events
5.4 Code workspace and evaluation	M9	Screens 4 and 5: file tree, viewer, diffs, test and quality results
5.5 Analytics	M9	Screen 6, analytics APIs and KPI calculations (Section 9.1)

Deliverables: full interactive dashboard with live updates.

Exit gate:

- [ ] A user can submit the demo task and watch every agent step live.
- [ ] Code differences between any two iterations are viewable.
- [ ] KPI values match figures computed directly from the database.

11.6 Phase 6 — Hardening, Evaluation and Release

Objective: make LoopForge reliable, measured and demonstration-ready.

Entry criteria: Phase 5 gate passed.

Work package	Modules	Description
6.1 Documentation Agent and final report	M10	README, API docs, Screen 7, downloadable artifacts
6.2 Budgets and human approval	M8	Cost budgets in the UI; approval gate for sensitive operations
6.3 Security review	M5	Prompt-injection tests, dependency and command allowlists, secret redaction
6.4 Benchmark suite	—	30–50 tasks with hidden tests; `run_benchmarks.py`
6.5 Experiments	—	E1–E3 comparisons and A1–A4 ablations with repeated trials
6.6 Release	—	Integration fixes, project documentation, demonstration rehearsal

Deliverables: benchmark results, evaluation report, complete documentation and a rehearsed demonstration.

Exit gate:

- [ ] All Section 15.1 acceptance criteria pass.
- [ ] Benchmark results are reported with mean, spread and cost for E1, E2 and E3.
- [ ] The end-to-end demonstration (Section 15.2) runs successfully from a clean environment.

11.7 Priority order under constraints

If time or budget is limited, preserve this order: (1) Planner → Builder → Verifier → Repair; (2) Decision Engine and retry logic; (3) independent evaluation; (4) Docker isolation; (5) persistent iteration history; (6) dashboard; (7) analytics and advanced reporting.

12. Testing and Evaluation Methodology

LoopForge is evaluated as an experiment, not a demo. The research question: **does a feedback-driven multi-agent workflow improve task completion and code quality compared with single-shot AI code generation, and does the gain justify the extra latency and cost?**

12.1 Platform test strategy

Level	Scope	Location
Unit	Controller decisions, schemas, scoring, error classifier	`backend/tests/unit/`
Integration	API endpoints, database repositories, sandbox lifecycle	`backend/tests/integration/`
Agent	Structured-output validity and failure handling per agent, using recorded LLM responses	`backend/tests/agents/`
Evaluation	Test runner, Ruff, Bandit and coverage parsers	`backend/tests/evaluation/`
End-to-end	Full task from submission to final report	Benchmarks (12.3)

The Decision Engine must reach 100% branch coverage, because every termination path is safety-relevant.

12.2 Experimental configurations

Experiment	Configuration
E1	Single-shot code generation
E2	Code generation plus one correction cycle
E3	Full LoopForge multi-agent iterative workflow

All configurations use the same LLM, task descriptions, test suites, execution environment and comparable token and execution budgets.

12.3 Benchmark dataset

An internal benchmark of 30–50 Python tasks across five categories, each with hidden acceptance tests, expected behaviour and a scoring rubric.

Category	Example task
CRUD development	Build employee management APIs
Authentication	Implement JWT-based authentication
Data validation	Build validated input processing
Bug fixing	Repair an existing broken API
Integration	Connect multiple API modules

Selected SWE-bench tasks can be added later for external credibility; full SWE-bench compatibility is a much larger undertaking.

12.4 Metrics

Report each metric individually; never fold them into one arbitrary improvement score.

Metric	Definition
Task success rate (TSR)	Successful tasks ÷ total tasks × 100
Test pass rate (TPR)	Passed tests ÷ executed tests × 100
Iteration efficiency (IE)	Successful tasks ÷ total iterations
Regression rate (RR)	Previously passing tests now failing ÷ previously passing tests × 100
Code coverage	Executed statements ÷ measurable statements
Execution time	Total time to complete a task
Token consumption	Total input and output tokens
Cost per success (CPS)	Total model and execution cost ÷ successful tasks

For the dashboard only, a composite quality score is used, with each component normalised to 0–100:

```
Q = 0.35\,T + 0.25\,R + 0.20\,C + 0.20\,S
```

where T is test correctness, R requirement satisfaction, C code quality and S security. The weights are design choices, not established values. Critical security findings are hard acceptance gates; a high average must never conceal them.

12.5 Results template

Populate only with measured values. No figures are reported until the benchmark has run.

Metric	E1	E2	E3
Task success rate	To be measured	To be measured	To be measured
Test pass rate	To be measured	To be measured	To be measured
Average iterations	1	To be measured	To be measured
Regression rate	To be measured	To be measured	To be measured
Cost per success	To be measured	To be measured	To be measured

12.6 Ablation studies

Study	Component removed	Question
A1	Independent Reviewer Agent	Does independent review improve quality?
A2	Persistent iteration memory	Does retained failure context reduce repeated mistakes?
A3	Requirement verification	Does explicit checking reduce incomplete implementations?
A4	Adaptive retry decisions	Does selective retry improve efficiency?

Because LLM output is stochastic, run repeated trials per task and report the mean, standard deviation, confidence intervals where appropriate, failure categories, and cost and latency trade-offs.

13. Security and Reliability Engineering

LoopForge executes AI-generated code, which creates a significant attack surface. The controls in this section are mandatory, not optional hardening.

13.1 Sandbox security

Generated code never runs on the host that serves FastAPI. Every sandbox container is configured with:

- [ ] Non-root user
- [ ] Read-only root filesystem where practical
- [ ] CPU, memory and process limits
- [ ] Execution timeouts
- [ ] Restricted filesystem mounts (task workspace only)
- [ ] No Docker socket mounted inside the sandbox
- [ ] No network access by default
- [ ] Explicit cleanup after execution

Ordinary containers are not a complete security boundary against hostile code. For stronger isolation, evaluate gVisor or Firecracker.

13.2 LLM and supply-chain security

Threat	Control
Prompt injection	Treat code, repository files, READMEs and test output as untrusted data; instructions found in them never override system policy
Malicious shell commands	Structured file-change outputs; command allowlist
Secret leakage	Secret redaction in logs and prompts; secrets never mounted into the sandbox
Dependency confusion and unsafe packages	Dependency allowlist; pinned versions
Unauthorised file modification	Writes confined to the task workspace; permission checks
Invalid agent output	Strict Pydantic schemas; reject and retry on validation failure
Sensitive operations	Human approval gate

13.3 Reliability controls

Risk	Mitigation
Infinite retry loops	Iteration limits
Excessive API usage	Token and cost budgets
Repeated identical failures	Failure fingerprinting and escalation
Agent timeout	Execution timeout and retry policy
Process or database interruption	Persistent LangGraph checkpoints
Regression after correction	Full regression suite on every iteration
False success	Independent final verification by a fresh test run

14. Technology Stack, Configuration and Deployment

Pin every dependency version in the real project; the table lists components, not versions.

14.1 Technology stack

Purpose	Library or tool
API framework	FastAPI
ASGI server	Uvicorn
Agent orchestration	LangGraph
LLM integration	OpenAI Python SDK
Validation	Pydantic
ORM	SQLAlchemy 2.x
Database driver	psycopg
Migrations	Alembic
Testing	Pytest
Coverage	Coverage.py

Purpose	Library or tool
Linting	Ruff
Security scanning	Bandit
Execution	Docker
Frontend	React + Vite
Workflow graph	React Flow
Charts	Recharts
HTTP client	Axios
Live updates	WebSocket

Redis and a background worker are introduced only when concurrent task execution becomes necessary.

14.2 Local deployment topology

Component	Runs as	Port
Frontend	React dev server	5173
Backend	FastAPI (Uvicorn)	8000
Database	PostgreSQL	5432
Sandbox	Isolated Docker containers, created per execution	None (no network)
LLM	OpenAI API (external)	HTTPS

Target environment: a developer machine running Windows with Docker Desktop, orchestrated by `docker-compose.yml`.

14.3 Environment configuration

`.env.example` (never commit the real `.env` or API keys):

```
DATABASE_URL=postgresql+psycopg://user:password@localhost:5432/loopforge
OPENAI_API_KEY=your_api_key
OPENAI_MODEL=your_supported_model
MAX_ITERATIONS=5
MAX_EXECUTION_SECONDS=120
MAX_TASK_DURATION_SECONDS=900
MAX_TOKEN_BUDGET=30000
ENVIRONMENT=development
```

14.4 Long-running execution

Tasks run outside the HTTP request. Start with FastAPI background orchestration or a dedicated worker process; for reliable long-running jobs, combine a persistent job queue with LangGraph checkpointing so a task resumes after a restart.

15. Acceptance Criteria and Demonstration

LoopForge is complete only when every criterion below is met with execution evidence.

15.1 Project acceptance checklist

- [ ] A user can create a project and submit a natural-language task.
- [ ] The Planner Agent produces a structured, schema-validated execution plan.
- [ ] The Builder Agent generates real Python files.
- [ ] Generated code executes in an isolated sandbox.
- [ ] The Verifier Agent runs real Pytest tests.
- [ ] Failures are captured and passed to the Repair Agent.
- [ ] The Repair Agent modifies the generated code.
- [ ] Tests re-run automatically after every correction.
- [ ] The Decision Engine enforces retry and resource limits.
- [ ] The Reviewer independently evaluates the implementation.
- [ ] Every iteration is stored in PostgreSQL.
- [ ] The dashboard shows live workflow progress.
- [ ] Users can inspect code differences between iterations.
- [ ] The system produces a final report with execution evidence.
- [ ] A failed or unsafe task is escalated, never falsely marked successful.

15.2 Demonstration scenario

Task: "Develop a FastAPI-based employee management system with authentication, CRUD endpoints, input validation and unit tests."

1. Enter the requirement into LoopForge.
2. The Planner Agent produces the execution plan.
3. The Builder Agent creates the application.
4. The Verifier Agent runs the tests.
5. The dashboard displays real failures.
6. The Repair Agent analyses and corrects them.
7. The next iteration runs automatically.
8. The Reviewer Agent checks the final implementation.
9. The Decision Engine accepts or escalates the result.
10. LoopForge generates the final engineering report.

The decisive moment is a genuine failure followed by an independently verified correction; that is what proves the feedback loop works.